

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 12 : Gestion du Temps et des Dates

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

12.0 Introduction : La notion du Temps pour un Microcontrôleur

Le temps relatif vs Le temps absolu

Par défaut, l'Arduino n'a aucune idée de l'heure qu'il est dans le monde réel. Il ne connaît que le "temps relatif" : combien de temps s'est écoulé depuis son dernier allumage.

Dans ce chapitre, nous allons apprendre à :

- Remplacer la fonction `delay()` pour créer des programmes multitâches.
- Utiliser les chronomètres internes : `millis()` et `micros()`.
- Mesurer la durée d'événements rapides.
- Introduire le temps absolu (Heure/Date) avec la bibliothèque `Time`.
- Utiliser un module matériel RTC (Real-Time Clock) pour sauvegarder l'heure même sans alimentation.

12.1 Le problème majeur de la fonction `delay()`

Depuis le début, nous utilisons `delay(1000)` pour faire des pauses d'une seconde.

Le Piège du Code Bloquant

`delay()` met le microprocesseur en pause totale. Pendant ce temps, l'Arduino **ne peut rien faire d'autre** : il ne peut pas lire un capteur, il ne peut pas détecter l'appui sur un bouton, il ne peut pas recevoir de message.

Conséquence : Impossible de faire clignoter une LED et de lire un bouton poussoir en même temps avec `delay()`. Il faut changer de paradigme.

12.2 Le chronomètre interne : `millis()`

La fonction `millis()` Retourne le nombre de millisecondes écoulées depuis que l'Arduino a démarré son programme actuel.

- C'est un compteur matériel tournant en tâche de fond.
- Le type de variable **doit impérativement** être `unsigned long` (entier long non signé de 32 bits).
- Pourquoi ? Parce qu'un simple `int` déborderait (reviendrait à zéro) au bout de 32 secondes. Avec un `unsigned long`, le débordement (Overflow) se produit après **environ 50 jours**.

12.3 Le Multitâche : "Blink Without Delay"

Le principe est de vérifier constamment l'heure, comme on regarde sa montre, sans s'arrêter de travailler.

```
const int ledPin = 13;
int ledState = LOW;
unsigned long tempsPrecedent = 0; // Stocke la derniere fois que la LED a change
const long intervalle = 1000;    // Intervalle de clignotement (1 seconde)

void setup() { pinMode(ledPin, OUTPUT); }

void loop() {
    unsigned long tempsActuel = millis(); // On regarde la "montre" de l'Arduino

    // Si la difference entre maintenant et la derniere action depasse l'intervalle :
    if (tempsActuel - tempsPrecedent >= intervalle) {
        tempsPrecedent = tempsActuel; // On met a jour l'heure de la derniere action

        // On inverse l'etat de la LED
        if (ledState == LOW) { ledState = HIGH; } else { ledState = LOW; }
        digitalWrite(ledPin, ledState);
    }

    // ICI, on peut ajouter du code pour lire un bouton sans etre bloqué !
}
```

12.4 Haute Précision : `micros()`

Parfois, la milliseconde n'est pas assez précise (ex : communication radio, capteurs à ultrasons).

La fonction `micros()` Retourne le nombre de **microsecondes** ($1\mu s = 10^{-6}s$) depuis le démarrage.

- Déborde et revient à zéro après environ **70 minutes**.
- Précision sur les cartes Arduino à 16 MHz (Uno, Mega) : la valeur retournée est un multiple de **4 microsecondes**.

L'algorithme de comparaison (`tempsActuel - tempsPrecedent`) fonctionne exactement de la même manière que pour `millis()`, et gère même nativement le débordement grâce aux mathématiques non signées.

12.5 Mesurer la durée d'un événement externe

Comment savoir combien de temps un utilisateur a maintenu un bouton appuyé ?

```
const int boutonPin = 2;
unsigned long tempsDebut = 0;
unsigned long dureeAppui = 0;

void setup() { Serial.begin(9600); pinMode(boutonPin, INPUT_PULLUP); }

void loop() {
    // Attend que le bouton soit presse (LOW)
    while(digitalRead(boutonPin) == HIGH) { /* Attente active */ }

    tempsDebut = millis(); // Enregistre l'heure de l'appui

    // Attend que le bouton soit relache (HIGH)
    while(digitalRead(boutonPin) == LOW) { /* Le doigt maintient le bouton */ }

    dureeAppui = millis() - tempsDebut; // Calcul de la duree totale
    Serial.print("Bouton maintenu pendant : ");
    Serial.print(dureeAppui);
    Serial.println(" ms");
}
```

12.6 La fonction `pulseIn()`

Pour mesurer des impulsions extrêmement courtes (générées par des capteurs de distance ou des récepteurs RC), l'Arduino propose une fonction native très optimisée.

Syntaxe : `duree = pulseIn(broche, ETAT, timeout);`

- Attend que la broche passe à l'ETAT défini (HIGH ou LOW), commence à chronométrer, puis s'arrête quand la broche change à nouveau.
- Retourne la durée de l'impulsion en **microsecondes**.
- *Attention* : `pulseIn()` est une fonction **bloquante** (comme `delay()`) tant que l'impulsion n'est pas terminée ou que le *timeout* n'est pas atteint.

12.7 Le temps absolu : La bibliothèque Time

Pour des projets comme un réveil, `millis()` ne suffit plus, il nous faut le format **Heure/Minute/Seconde/Jour/Mois/Année**.

La bibliothèque `TimeLib.h` (de Paul Stoffregen) : Elle simule une horloge (RTC logicielle) en utilisant les timers de l'Arduino.

Problème majeur : L'Arduino n'a pas de pile de sauvegarde interne. À chaque coupure de courant ou appui sur le bouton RESET, l'horloge repartira à la date de compilation ou au 1er Janvier 1970 (UNIX Epoch).

12.8 Le module RTC matériel (Real-Time Clock)

Pour garder l'heure exacte pendant des années, on utilise un composant dédié avec sa propre petite pile ronde (CR2032).

Les deux standards de l'industrie :

- **DS1307** : Le plus ancien. Bon marché, mais sa précision dérive légèrement avec les changements de température (peut perdre quelques minutes par mois).
- **DS3231** : Le moderne. Intègre un capteur de température interne (TCXO) pour ajuster le cristal de quartz. Précision extrême (dérive de moins d'une minute par an).

[Insérer l'image d'un module RTC DS3231 avec sa pile bouton]

12.9 Câblage du module RTC

Les modules RTC (DS1307 ou DS3231) communiquent avec l'Arduino via le bus **I2C**.

Connexions (4 fils) :

- **VCC** : 5V (Généralement, bien que le DS3231 tolère 3.3V).
- **GND** : Masse.
- **SDA (Données)** : Broche A4 sur Uno / Broche 20 sur Mega.
- **SCL (Horloge)** : Broche A5 sur Uno / Broche 21 sur Mega.

*Le bus I2C permet de brancher l'écran OLED (Chapitre 11) et le module RTC sur les **mêmes broches** (A4/A5) simultanément, car ils ont des adresses différentes !*

12.10 La bibliothèque RTCLib

Pour utiliser ces modules facilement, la bibliothèque de référence est RTCLib (par Adafruit).

```
#include <Wire.h>
#include "RTCLib.h"

RTC_DS3231 rtc; // Creation de l'objet RTC (ou RTC_DS1307)

void setup() {
  Serial.begin(9600);
  if (! rtc.begin()) {
    Serial.println("Horloge introuvable !");
    while (1); // Bloque le programme
  }

  if (rtc.lostPower()) {
    Serial.println("Le RTC a perdu l'heure, réglage...");
    // Regle le RTC sur la date et l'heure de compilation du code !
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }
}

void loop() {}
```

Une fois le RTC initialisé, on lit l'heure en appelant la méthode `now()`.

```
void loop() {  
    DateTime maintenant = rtc.now(); // Recupere l'heure actuelle du module  
  
    Serial.print(maintenant.day(), DEC);    Serial.print('/');  
    Serial.print(maintenant.month(), DEC); Serial.print('/');  
    Serial.print(maintenant.year(), DEC);   Serial.print(" - ");  
  
    Serial.print(maintenant.hour(), DEC);   Serial.print(':');  
    Serial.print(maintenant.minute(), DEC); Serial.print(':');  
    Serial.print(maintenant.second(), DEC); Serial.println();  
  
    delay(1000); // Ici on peut se permettre un delay si on ne fait rien d'autre  
}
```

12.11 Formater l'heure avec les zéros (ex : 09 :05)

Problème esthétique : L'Arduino affichera 9:5 au lieu de 09:05. Il faut corriger cela manuellement ou utiliser `sprintf()`.

```
void loop() {
    DateTime maintenant = rtc.now();
    char buffer[20];

    // %02d signifie : entier decimal sur 2 positions, rempli avec des zeros
    sprintf(buffer, "%02d/%02d/%04d %02d:%02d:%02d",
            maintenant.day(), maintenant.month(), maintenant.year(),
            maintenant.hour(), maintenant.minute(), maintenant.second());

    Serial.println(buffer); // Affiche "05/04/2026 09:05:03"
    delay(1000);
}
```

12.12 Créer des alarmes simples

Avec le temps absolu, on peut facilement déclencher des relais (vu au Chapitre 10) à des heures spécifiques de la journée (ex : arrosage automatique).

```
const int pompeRelais = 7;

void loop() {
    DateTime maintenant = rtc.now();

    // Si on est a 20h00 precises (et 0 seconde pour ne l'allumer qu'une fois)
    if (maintenant.hour() == 20 && maintenant.minute() == 0 && maintenant.second() == 0)
    {
        digitalWrite(pompeRelais, LOW); // Allume la pompe

        // Attente bloquee volontaire de 10 minutes d'arrosage
        // Pour du vrai multitache, il faudrait utiliser la methode millis() ici !
        delay(600000);

        digitalWrite(pompeRelais, HIGH); // Eteint la pompe
    }
}
```

Résumé du Chapitre 12

- **Le bannissement de `delay()`** : Pour un code robuste et interactif, apprenez le concept "Blink Without Delay" basé sur la soustraction de `millis()`.
- **Variables** : Utilisez toujours `unsigned long` pour manipuler `millis()` ou `micros()` afin de gérer proprement les dépassements de mémoire.
- **Temps Absolu** : L'Arduino ne retient pas l'heure. Un projet nécessitant l'heure réelle a obligatoirement besoin d'une connexion internet (NTP) ou d'un module **RTC**.
- **Le module DS3231** : C'est le composant I2C de référence à privilégier pour sa grande précision thermique et sa fiabilité.

Fin du Chapitre 12. Le Chapitre 13 aborde en profondeur les protocoles I2C et SPI (Communicating Using I2C and SPI).